

Bocconi

COMPUTER SCIENCE

code 30424 a.y. 2024-2025

Lesson 19

Classes, instances and access to files



Università
Bocconi
MILANO

Objectives of the lesson

- Understand the concepts of classes, attributes and methods
- Learn how to access files

1.

Classes, attributes and methods



From Variables to Objects

- Python supports different types of data:

1946

3.1415926535

"Hello World!"

[1, 5, 7, 11]

(1, 5, 7, 11)

- A variable can be used to represent a value of a specific type, but it can also be used to perform specific actions on that type of value
 - `MyString.upper()`
 - `MyList.append(...)`
 - `MyDict.items()`
 - Etc.
- The actions that each variable can perform depend on the type of data to which it belongs
- In Object Oriented languages (like Python) elements that can represent a value and that can perform actions are defined as **Objects** (i.e. variables are all objects)

Classes of objects

- Every type of variable has different features both in terms of the values that it can represent and in terms of the actions that can be performed with it
- All variables of a specific type have the same features
- In Object Oriented languages (like Python) the collection of features of a specific type of variable is defined as **Class**

Instances

- When a variable is initialized, a specific **Object** of a specific **Class** is created
- In Object Oriented languages (like Python) a specific Object of a specific Class is defined as an **Instance**

`MyNum = 125` → `MyNum` is an Instance of the Class `int`

`a = "Hello World!"` → `a` is an Instance of the Class `string`

`MyList = [1,2,3,4]` → `MyList` is an Instance of the Class `list`

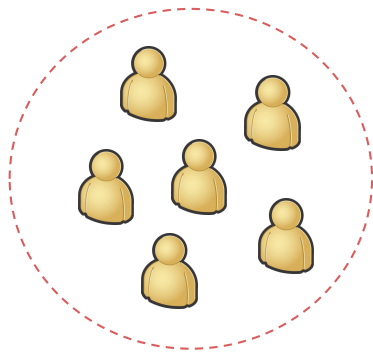


Attributes e methods

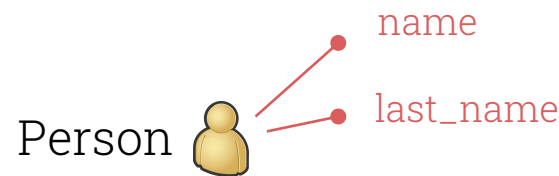
- A Class is defined by its name and by a set of attributes and methods
- Attributes are the features of the Class:
 - The content of a string is the attribute «**value**» of the class string
- Methods are the actions that Objects of a Class can perform:
 - **upper** is a method of the class string
 - **append** is a method of the class list

To summarize

- **Object:** it is an «element» characterized by attributes (how it is made) and methods (how we can modify and use it)
- **Class:** is the abstract description of an object. It represents the «mold» used to create all objects characterized by the same set of attributes and methods that define the class
- **Instance:** is the term used for a specific «real» object of a class



Group of objects of
the same class



Class: it is used to
create objects with
common features



Instance: a specific object
created from a class

Creation of a class

- To create a class you need to give it a name and define its attributes and/or methods
- All instances of the class inherit attributes and methods from the class to which they belong
- Attributes can be assigned in the creation phase of the instance and can be modified later

Creation of the class Person

definition of the class

name of the class

DEFINITION OF A CLASS OF OBJECTS



```
class Person():
```

```
    ''' Class of objects of type: Person '''
```

```
    attribute
```

```
    attribute
```

```
    attribute
```

attributes of the class

```
def method(self):
```

```
    instruction
```

```
    instruction
```

method of the class

Attributes of a class

- They allow to customize each single object of the class (i.e. the instances)
- They can be Python built-in types (i.e.: *int*, *float*, *list*) or other objects
- They can be defined directly in the class (with an assignment, as shown here below), but they are usually defined with a special method called *constructor method* (see next slide)

```
class Person():  
    name = ""           # str, initially null  
    surname = ""        # str, initially null  
    height = 0          # int, cm, initially zero  
    weight = 0.0         # float, kg, initially zero
```



Constructor method `__init__`

- It is the method that Python searches in the class when it initializes the state of a new instance
- It can include the attributes definition and any other feature and option needed to create a new instance, e.g. calling a function, opening files, access to databases, etc.
- The syntax follows the same rules used when defining a function, with the parameters (mandatory or optional) used to initialize the attributes specified in parentheses
- There must be at least one parameter – conventionally named **self** – that represent the reference to the name of the instance that will be created

Creation of `__init__` method

```
class Person():  
    def __init__(self, nat, FirstName="", LastName=""):  
        self.name = FirstName  
        self.surname = LastName  
        self.nationality = nat  
        self.height = 0  
        self.weight = 0.0
```

initialization method

'self' parameter

mandatory parameter

optional parameters

Initialization of attributes not defined as parameters
(see examples in following slides)



Creation of instances of the class Person

name of the instance

calls the `__init__` method that creates an instance of the class «Person» with the attributes specified as parameters

```
MyPresident = Person('english')
```

```
MyDirector = Person('mexican')
```

```
MyAssistant = Person('french')
```

INSTANCE AN OBJECT STARTING FROM ITS DEFINITION

MyPresident



MyDirector



MyAssistant



Interact with object attributes

- To access a specific object, we must use the name of its instance
- There are two different ways to define the values of attributes:

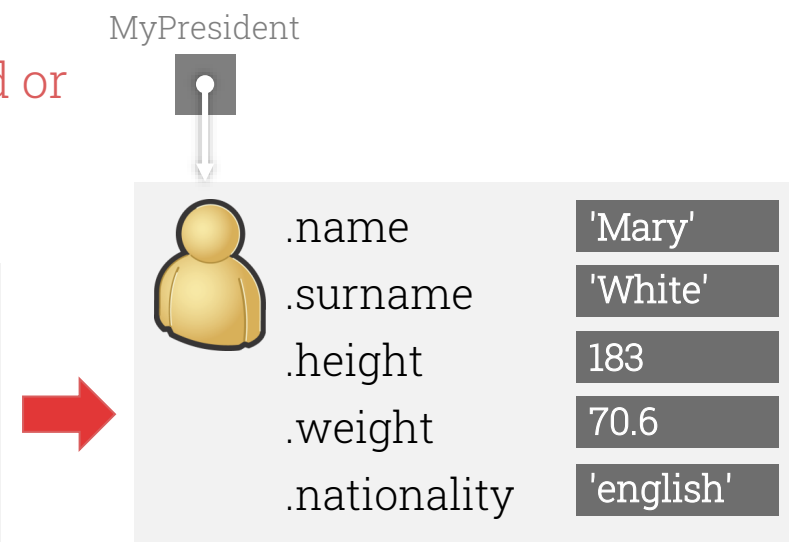
InstanceName = ClassName(val_1, val_2, ...)

(only if the attribute can be initialized by the constructor method `__init__`)

InstanceName.AttributeName = value

(for attributes that cannot be initialized by the constructor method or to modify attributes already initialized by `__init__` method)

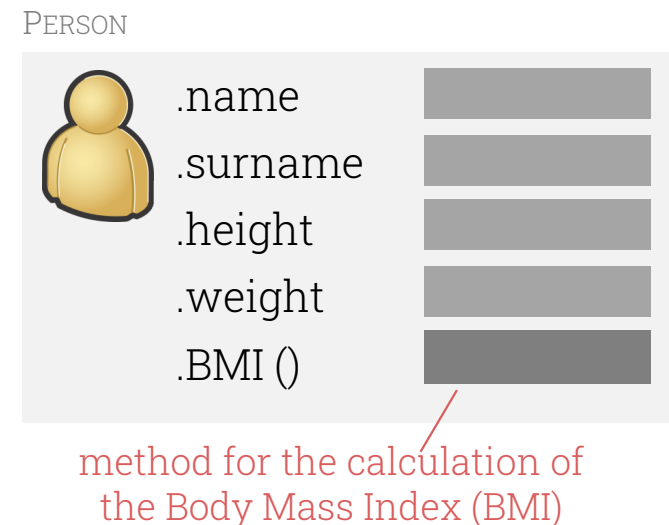
```
MyPresident = Person('english', 'Mary', 'White')  
MyPresident.height = 183  
MyPresident.weight = 70.6
```



Methods

- They are not variables/objects that contain a value, but procedures that execute instructions
- They can return a value thanks to the **return** statement
- They have at least one parameter, named **self**
- They are invoked with the following syntax:

`InstanceName.MethodName(par_1, par_2, ...)`



Methods of the class

```
class Person():
```

```
    def __init__(self, nat, FirstName="", LastName=""):  
        self.name = FirstName  
        self.surname = LastName  
        self.nationality = nat  
        self.height = 0  
        self.weight = 0.0
```

mandatory parameter for all methods:
in this case, it represents the future
instances of class Person

self.weight and *self.height* identify the attributes of the object

```
    def BMI(self):  
        bmi_value = self.weight / (self.height/100)**2  
        return bmi_value
```

keyword to define
a method

keyword to allow a
method to return a value

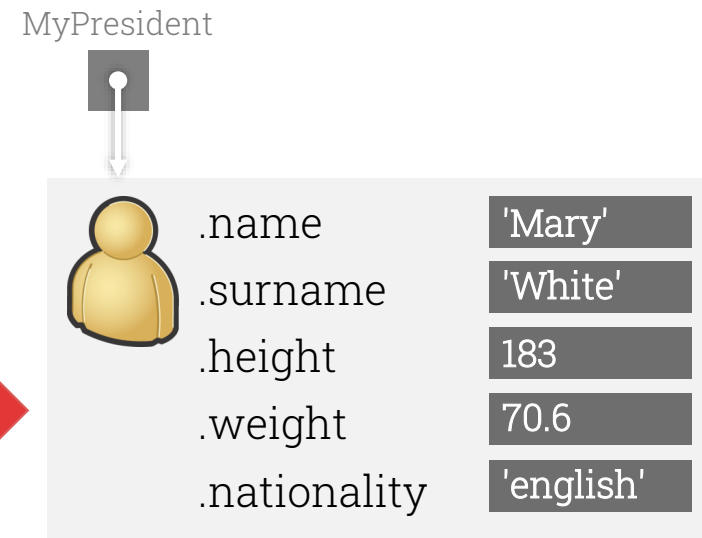
formula used to calculate the body mass index, to
be returned by the method



Example

Once created (in the Editor) the script that define the class **Person** (see previous slide), we can run it and we can create the instance **MyPresident** in the shell:

```
MyPresident = Person('english', 'Mary', 'White')
MyPresident.height = 183
MyPresident.weight = 70.6
```



Now, we can invoke the **BMI** method and print its output with the following instruction:

```
>>> print(MyPresident.BMI())
21.08154916539759
>>>
```

Special method `__str__`

What happens if we use the name of an instance as argument of a print function?

- The printout will be the result of the special method `__str__`
- `__str__` has no arguments (besides **self**) and must end with a **return** statement

```
class Person:

    def __str__(self):
        t = (self.name + " " +
              self.surname + " (" +
              str(self.height) + " cm, " +
              str(self.weight) +
              " kg, has a Body Mass Index of " +
              str(round(self.BMI(), 1)) + ')')
        return t
```

MyPresident



	.name	"Mary"
	.surname	"White"
	.height	183
	.weight	70.6
	.nationality	'english'

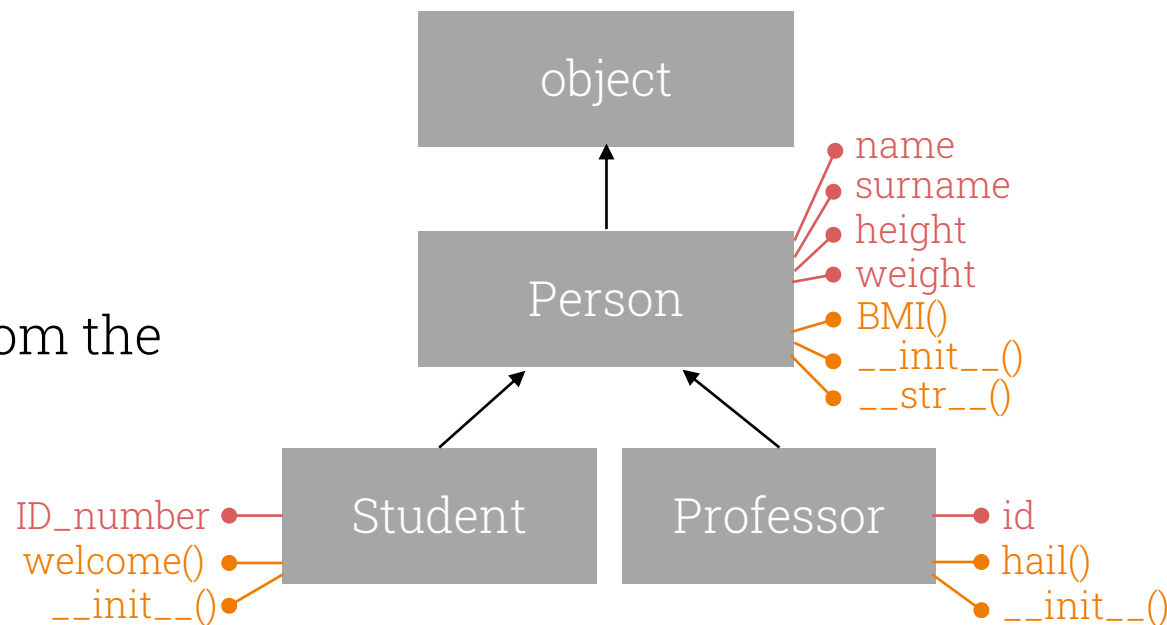
`>>> print(MyPresident)`

Mary White (183cm, 70.6kg, has a Body Mass Index of 21.1)



Hierarchies and Inheritance

- **parent class** (superclass)
- **child class** (subclass)
 - **inherits** all attributes and methods from the parent class
 - **adds** more attributes and methods
 - **overwrites** attributes and methods of the parent class



```
class Person():  
class Student(Person):  
class Professor(Person):
```

subclass of the objects category

superclass for Student and Professor

subclasses of Person



Creation of a child class

`class Student(Person):` ————— The parent class Person must be defined before its child class Student

```
def __init__(self, IDnum, nat, FirstName="", LastName=""):
    Person.__init__(self, nat, FirstName, LastName)
    self.ID_number = IDnum
```

Initialization makes use of the `__init__` of the parent class, but it also adds a new attribute

The `__str__` method of the superclass is completely redefined here

```
def __str__(self):
    t = (self.name + " " +
         self.surname + " - " +
         "student ID " +
         self.ID_number)
    return t
```

`def welcome(self):` ————— This is a new method of the child class

```
w = ("Hello, my name is " +
     self.name)
return w
```



Use of a child class

- Creating an instance of the parent class **Person**:

```
>>> MyDirector = Person('mexican', 'Juan Esteban', 'Gonzales')
>>> MyDirector.height = 169
>>> MyDirector.weight = 71.4
>>> print(MyDirector)
Juan Esteban Gonzalez (169 cm, 71.4 kg, has a Body Mass Index of 25.0)
```

The `__str__` of class
Person is used here

- Creating an instance of the child class **Student**:

```
>>> MyMate = Student('123456', 'italian', 'Mario', 'Rossi')
>>> MyMate.height = 169
>>> MyMate.weight = 71.4
>>> print(MyMate)
Mario Rossi - student ID 123456
>>> print("The Body Mass Index of", MyMate.name, "is", format(MyMate.BMI(), ".1f"))
The Body Mass Index of Mario is 25.0
>>> print(MyMate.welcome())
Hello, my name is Mario
```

The `__str__` of the class
Student is used here

We can use the new
methods of the child class
Student

Student inherited the methods
of the class Person



2. Accessing files

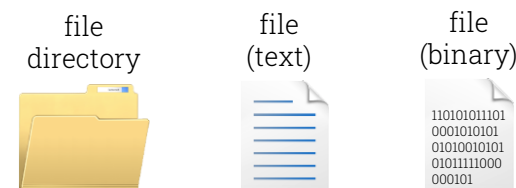


Access to files

To access a file in Python we use the built-in function `open`, which:

- opens a file and returns an **object** of type **file** which is linked to the file contents
- raises an error if the file cannot be opened

The **file** object returned by `open` must be assigned to a variable.



The built-in function open

- The syntax:

```
open(file, mode)
```

Mandatory argument: a text string that specifies the name of the file, including its complete path in the disk

- To open a file in read-only mode:

```
f = open('G:\\Data\\students.txt', 'r')
```

Optional argument: it specifies how the file should be opened (in reading or writing mode). If the argument is omitted, the file is opened as read-only

Opens the file in read-only mode (default mode)

- To read the content of a text file:

```
row = f.readline()
print(row)
```

```
>>> Mario Rossi (183cm, 81.5kg)
>>>
```

- To close (and save) a file:

```
f.close()
```

data.txt



```
first row
second row
third row
```

```
...
EOF
```

The mode argument of the open function

Mode	Description
'r'	The file is opened in read-only mode. It's not possible to modify its content or enter new data. This is the default mode if the mode argument is omitted. When the file is opened, Python is positioned at the beginning of the first row (position 0). If the file doesn't exist, Python returns an error.
'w'	The file is opened in write mode. If the file already exists, the data contained in it is deleted. If the file doesn't exist, a new file is created.
'a'	the file is opened in append mode. When the file is opened, Python is positioned at the end of the last row, therefore all data that Python writes in the file are added after the existing content. If the file does not exist, a new file is created and Python is positioned at the beginning of the file.
'+'	The «+» symbol after r , w or a introduces a double access mode (read and write).



Reading methods

The content of a file can be read entirely or line by line:

- The **read** method reads the content of the entire file
- The **readline** and **readlines** methods read the content line by line

```
f = open('students.txt')
```

```
row = f.readline()
```

```
while row != '':
```

```
    print(row)
```

```
    row = f.readline()
```

```
f.close()
```



Writing in a text file

```
f = open("students.txt", "w")
```

```
f.write("Anna Corti\n")
```

```
f.write("Salvo Brandi\n")
```

```
f.write("Nina Ferdi\n")
```

```
f.close()
```

Opens the **students.txt** file in write mode. If the file already exists, the content is deleted. If the file does not exist, it is automatically created.

To end a line of text when writing in a file, the "new line" character is needed.

```
f = open("students.txt", "a")
```

```
f.write("Lola Manno\n")
```

```
f.close()
```

Opens the **students.txt** file in append mode so that the new line of text will be added at the end of the file.

Exercise 1

The file you need to use is called **Exercise 1.py** and contains a program written in Python aimed to solve some geometric problems. The program is incomplete and contains the definition of the class **Circle**.

You must write the code necessary to complete the program so that:

- it defines the class **Square** with the attribute *side*
- it defines the class **Rectangle** with the attributes *base* and *height*
- it defines, for each of the two classes, a method to calculate the area
- it defines, for the **Rectangle** class, a method to calculate the diagonal of the rectangle using the Pythagorean theorem:

$$diagonal = \sqrt{base^2 + height^2}$$

Exercise 1

Now create a function, called **largest_area**, to calculate the largest area of three objects. The function must:

- have three instances as mandatory parameters
- calculate the area of each instance
- show the text 'Largest area = XXX' to print the value of the largest area on the screen

Complete the program creating an instance for each class, e.g.:

- an instance of the **Circle** class with a radius of 9
- an instance of the **Square** class with side of 10
- an instance of the **Rectangle** class with base 7.5 and height 12

Then, call the **largest_area** function to find the value of the largest area among those of the three instances passed to the function.

Tools to learn with today's lecture

- Understand the meaning of "object"
- Know the difference between "class", "object" and "instance"
- Learn how to define the attributes of an object
- Define new classes, instantiate new objects, copy and modify existing objects
- Define new methods
- Understand the concept of "inheritance"
- Learn how to read data from files and how to write data into files using the built-in function `open`

Book references

Understanding Python:

Chapters 10.1, 10.2, 10.3, 11

Python workbook:

Exercises of Units 9, 11